Searching.

DS 2019/2020

The search problem

- The static aspect:
  - U the universe set, $S \subseteq U$
  - the search operation:
    - Instance: $a \in U$
    - Question: $a \in S$?

Ex: U = natural numbers
S = all numbers from our vector

Question is, a is in our vector?
(a is in U but is a in S?)

- The dynamic aspect:
  - the insert operation
    - Input: S, $x \in U$
    - Output: $S \cup \{x\}$

The same example, U is all natural numbers, S is the reunion with the element x that is a natural no.

  - the delete operation
    - Input: S, $x \in U$
    - Output: $S - \{x\}$

Exactly as above but S is the set without x, hence the deletion.

Content

~~The search problem~~

Binary search

Binary search trees

Balanced search trees

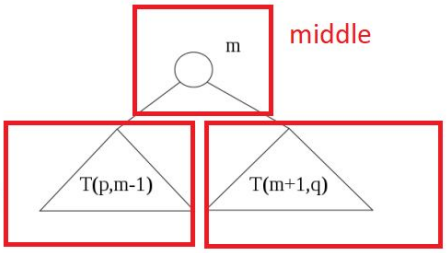
Binary search: the static aspect

- The universe set is totally ordered: $(U, \leq)$ Translation: the vector must be sorted

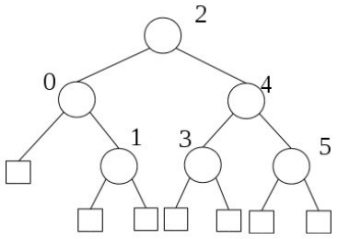
  Totally ordered=ordered set without duplicates, ex: 1,2,3 is totally ordered but 1,1,2 is not.
- The used data structure:
  - the array $s[0..n-1]$
  - $s[0] < ... < s[n-1]$

```
Function pos(s[0..n − 1], n, a)
begin
    p ← 0; q ← n − 1
    m ← (p + q)/2
    while (s[m]! = a and p < q) do
        if (a < s[m]) then
            q ← m − 1
        else
            p ← m + 1
        m ← (p + q)/2
    if (s[m] = a) then
        return m
    else
        return −1
end
```

$T(p, q)$



$T = T(0, n-1)$
$n = 6$



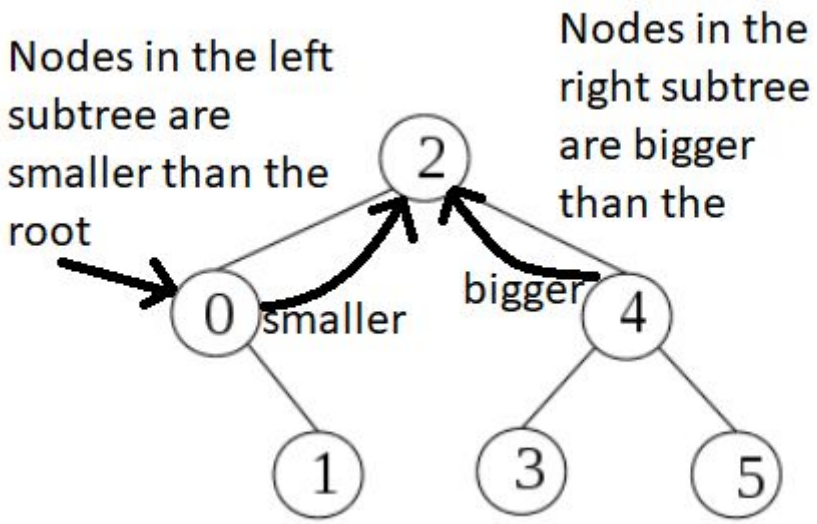
Content

~~The search problem~~

~~Binary search~~

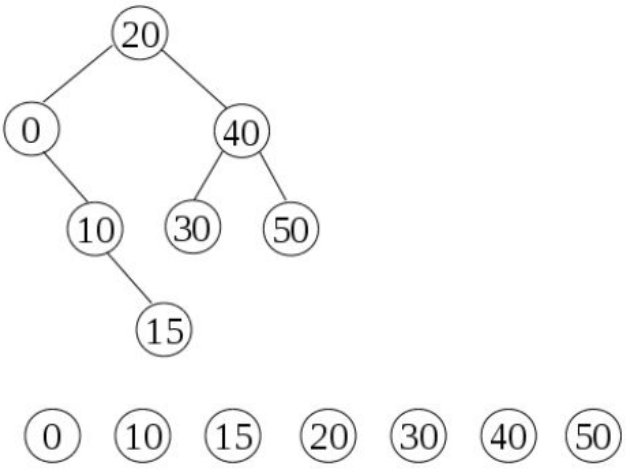
Binary search trees

Balanced search trees



- Inorder traversal

Function *inorder(v, visit)*
begin
  if *(x == NULL)* **then**
    return
  else
    inorder($v \rightarrow stg$, visit)
    visit(v)
    inorder($v \rightarrow drp$, visit)
end



- Time complexity: $O(n)$

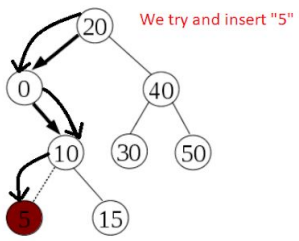
Binary search trees: searching

Function $poz(t, x)$
begin
  $p \leftarrow t$
  while $(p! = NULL \text{ and } p \rightarrow val! = x)$ **do**
    if $(x < p \rightarrow val)$ **then**
      $p \leftarrow p \rightarrow stg$
    else
      $p \leftarrow p \rightarrow drp$
  return p
end

- Time complexity: $O(h)$, h the height

Binary search trees: insertion

- Search in the tree the place to insert the new element (similarly with the search operation).
- Add the node with the new information, and the left subtree, respectively the right one is NULL.



Procedure $insBinarySearchTree(t, x)$
begin
  if $(t == NULL)$ **then**
    new(t); $t \rightarrow val \leftarrow x$; $t \rightarrow stg \leftarrow NULL$; $t \rightarrow drp \leftarrow NULL$
  else
    $p \leftarrow t$
    while $(p! = NULL)$ **do**
      $predp \leftarrow p$
      if $(x < p \rightarrow val)$ **then** $p \leftarrow p \rightarrow stg$;
      else
        if $(x > p \rightarrow val)$ **then** $p \leftarrow p \rightarrow drp$;
        else $p \leftarrow NULL$;
    if $(predp \rightarrow val! = x)$ **then**
      if $(x < predp \rightarrow val)$ **then**
        /* add x as left child of $predp$ */
      else /* add x as right child of $predp$ */ ;
end

Binary search trees: elimination

Multiple cases, we will not go through them here

Search x in the tree t; if it is found, then distinguish the following cases:

- Case 1: the node p which stores x has no children;
- Case 2: the node p which stores x has a single child;
- Case 3: the node p which stores x has both children.
  - Find the node q which stores the highest value y smaller than x (get down from p to the left and then to the right as much as possible).
  - Interchange the values from p and q.
  - Delete q as in case 1 or 2.

Time complexity: $O(h)$, h the height.

Content

~~The search problem~~

~~Binary search~~

~~Binary search trees~~

Balanced search trees

Balanced search trees

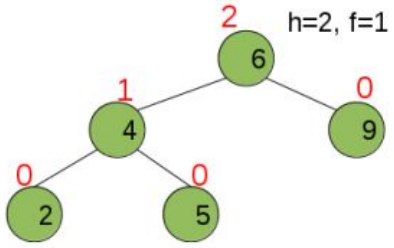
- AVL trees (Adelson-Velsii and Landis, 1962) — We will discuss them
- B trees/2-3-4 trees (Bayer and McCreight, 1972) — Used in databases, 2nd year
- Red-black trees (Bayer, 1972) — We will discuss them
- Splay Trees (Sleator and Tarjan, 1985)
- Treaps (Seidel and Aragon, 1996)

AVL trees (G. **A**delson-**V**elskii, E.M. **L**andis 1962)

- A binary search tree t is a balanced **AVL tree** if for each vertex v,

$$|h(v \to stg) - h(v \to drp)| \leq 1$$

- $h(v \to stg) - h(v \to drp)$ is called the **balancing factor**.
- Example:



- **Lemma**

  If t is an AVL tree with n internal nodes then $h(t) = \Theta(\log n)$.

AVL trees

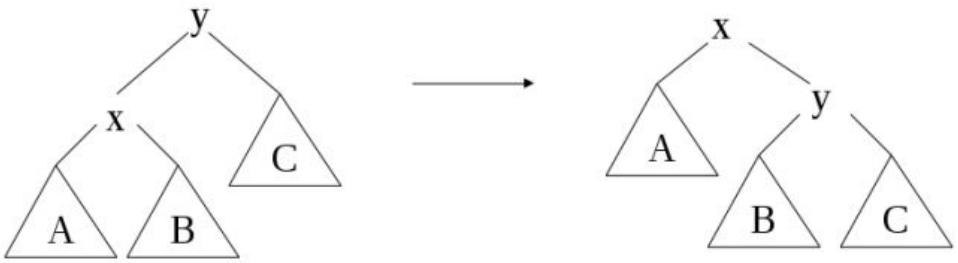
- **Theorem**

  The class of AVL trees is $O(\log n)$ stable.

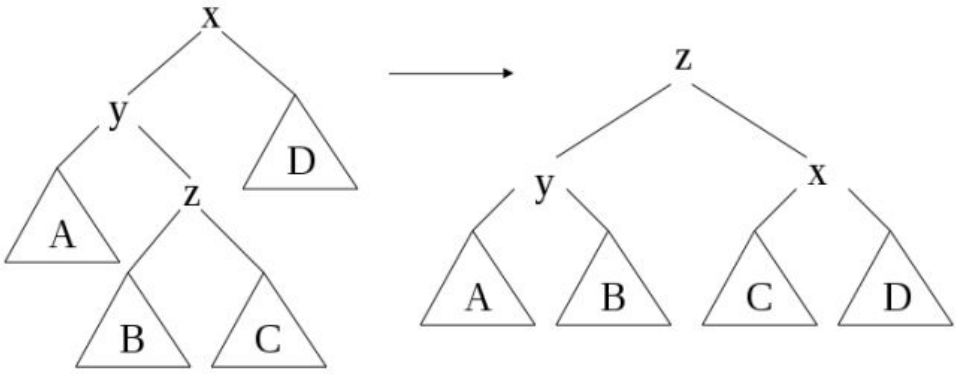
- The search/deletion algorithm
  - The nodes have also saved the balancing factors $(-1, 0, 1)$.
  - Store the path from the root to the added/deleted node in a stack ($O(\log n)$).
  - Traverse the path stored in the stack in reverse order and rebalance the unbalanced nodes with one of the operations: left/right rotation simple/double ($O(\log n)$).

Rotations We will not discuss them, please study from course

Right rotation (simple)



Double right rotation



Advantages/drawbacks of AVL trees

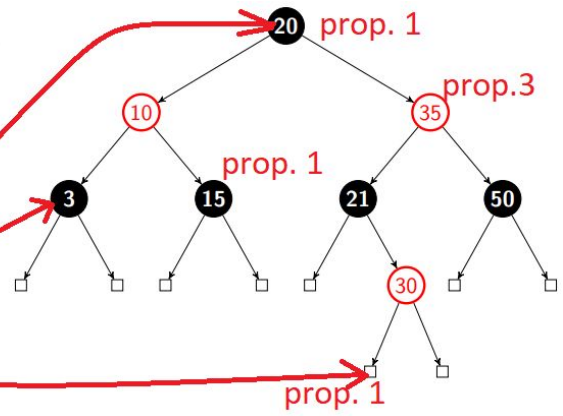
- Advantages:
  - Searching, insertion and deletion takes $O(\log n)$ complexity.
- Drawbacks:
  - Additional space for storing the height / the balancing factor.
  - The re-balancing operations are expensive.
- Are favorite when we are making more searches and fewer insertions and deletions
- Applications in Data Analysis, Data Mining

Balanced search trees.

DS 2019/2020

Red-black trees

- *Symmetric binary B-tree*, Rudolf Bayer, 1972.
- The balancing is maintained by using a coloring of the nodes.
- The red-black trees are binary search trees that satisfy the following properties:
  1. a node is colored with red or black;
  2. the root and the leaf nodes (*nil* – that belong to the structure) are colored with black;
  3. if a node is red, then his both children are black;
  4. the paths from a node to the boundary nodes have the same number of black nodes.

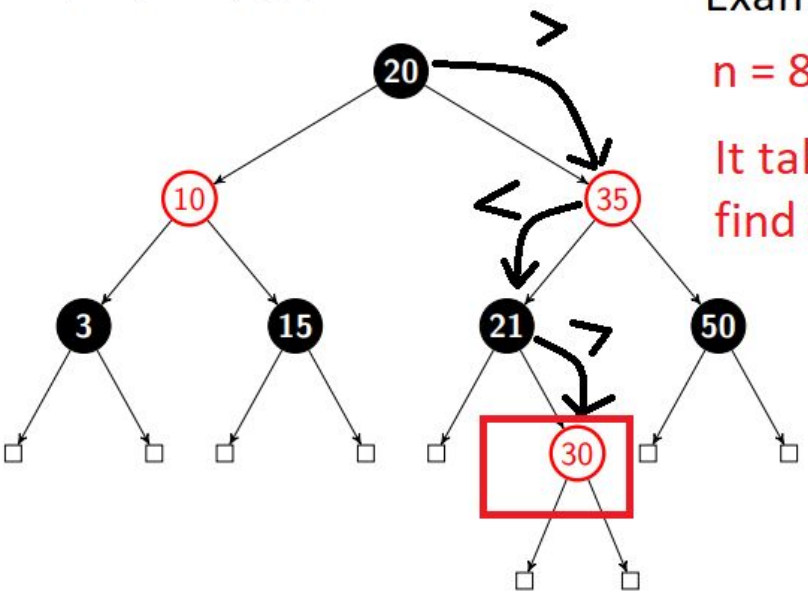


In a red-black tree with n nodes, the search operation has a time complexity of $O(\log n)$.

Example: We search for 30

n = 8, log2(8) = 3

It takes 3 opperations to find 30, hence O(log n)



PLEASE CHECK IN THE COURSE THE FOLLOWING OPERATIONS:

- INSERT
- DELETE
- REBALANCE → VERY IMPORTANT!

SINCE THEY ARE QUITE COMPLEX WE DO NOT HAVE TO GO OVER THEM AGAIN.